

EPIC EVENTS

Technical architecture and security

Command-line CRM - Python, Peewee, SQLite, Typer, JWT, Sentry and Pytest

Automated tests	156
Failures	0
Total coverage	77%
JWT coverage	96%

OpenClassrooms Project P12 - July 2026

Contents

1. Validation summary
2. Architecture overview
3. Database schema
4. JWT authentication and authorization
5. Security and Sentry monitoring
6. Testing strategy and coverage
7. Installation and demonstration
8. Final repository verification

1. Validation summary

Epic Events implements a layered architecture, a relational SQLite database, JWT authentication, authorization based on role, ownership and assignment, least privilege, Sentry monitoring, and an automated test suite.

- 156 tests passed, 0 failures.
- 77% total coverage; 96% for src/core/jwt_auth.py.
- 4 unit tests, 143 integration tests, and 9 functional tests.
- In-memory isolated SQLite test database and temporary JWT test files.
- Secrets, local token, local database, and generated reports excluded from Git.
- Controlled demonstration event successfully received by Sentry.

Metric	Value
Tests	156
Failures	0
Total coverage	77%
JWT coverage	96%

2. Architecture overview

2.1 Purpose

The CRM is used by management, sales, and support departments. The architecture separates terminal interaction, authentication session management, business rules, persistence, and monitoring.

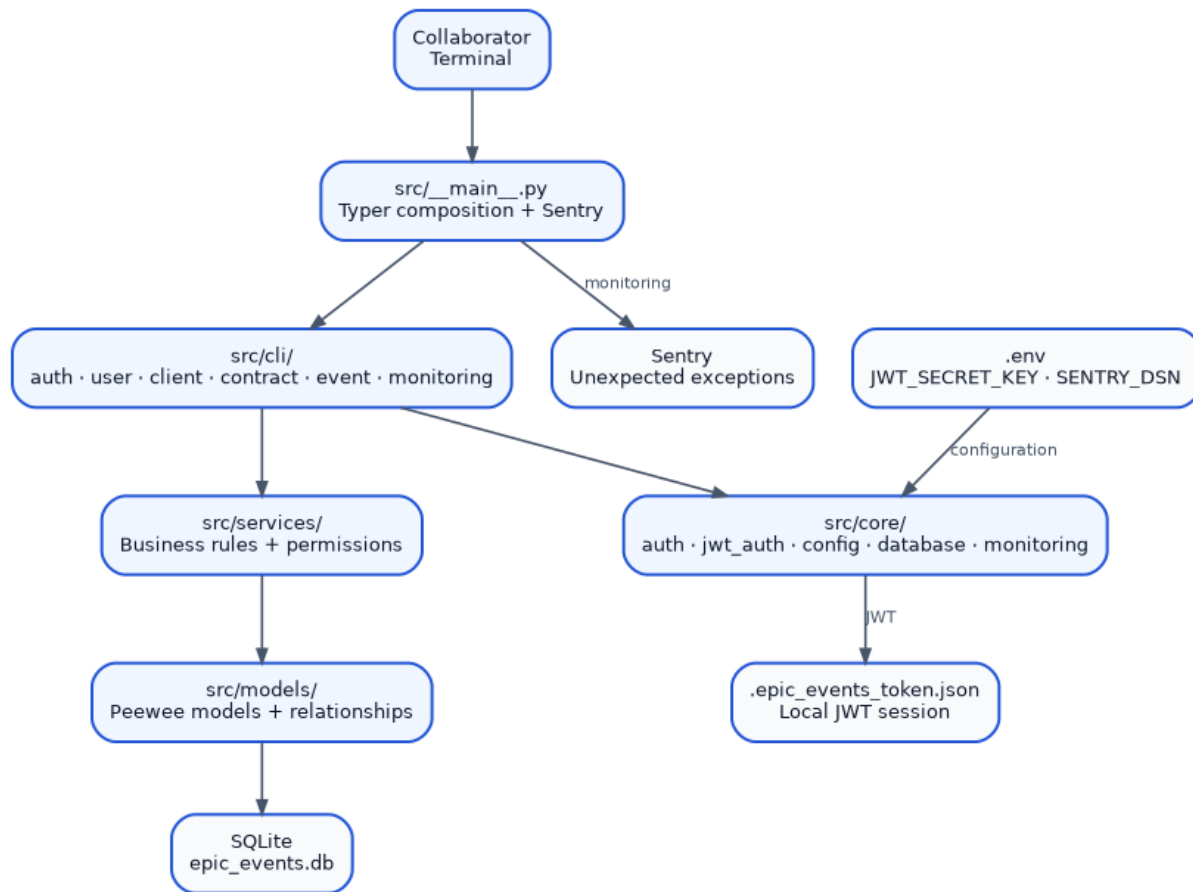


Figure 1 - Current layered architecture

2.2 Layer responsibilities

Component	Responsibility
src/__main__.py	Composes Typer groups, initializes Sentry, and catches unexpected exceptions.
src/cli/	Collects arguments, displays results, and converts business errors into CLI responses.
src/core/auth.py	Verifies bcrypt credentials and exposes generic department checks.
src/core/jwt_auth.py	Creates, signs, validates, loads, stores, and deletes JWTs.
src/services/	Enforces permissions, ownership, assignment, and business validation.
src/models/	Defines the Peewee schema and relationships.
src/utils/	Initializes the database and the first management account.
Sentry	Receives unexpected technical exceptions and the controlled demonstration.

2.3 Project structure

```
src/
├── __main__.py
├── cli/
│   ├── auth.py  client.py  common.py
│   ├── contract.py  event.py  monitoring.py  user.py
├── core/
│   ├── auth.py  config.py  database.py
│   ├── jwt_auth.py  monitoring.py
├── models/
├── services/
├── utils/
│   ├── create_db.py
│   └── create_user.py
```

The entry point no longer contains business rules. Commands are grouped by domain with `app.add typer` and the application runs with `python -m src`.

2.4 Protected command flow



Figure 2 - Authentication followed by authorization

3. Database schema

3.1 SQLite and Peewee choice

SQLite fits a local command-line application: no database server is required and installation remains reproducible.

Peewee provides model-based schema definitions, foreign keys, and parameterized queries.

- Foreign keys enabled through the `foreign_keys` pragma.
- `RESTRICT` protects referenced clients, contracts, and sales collaborators.
- `SET NULL` preserves an event when its support reference is removed.
- Logical account deactivation preserves CRM history.

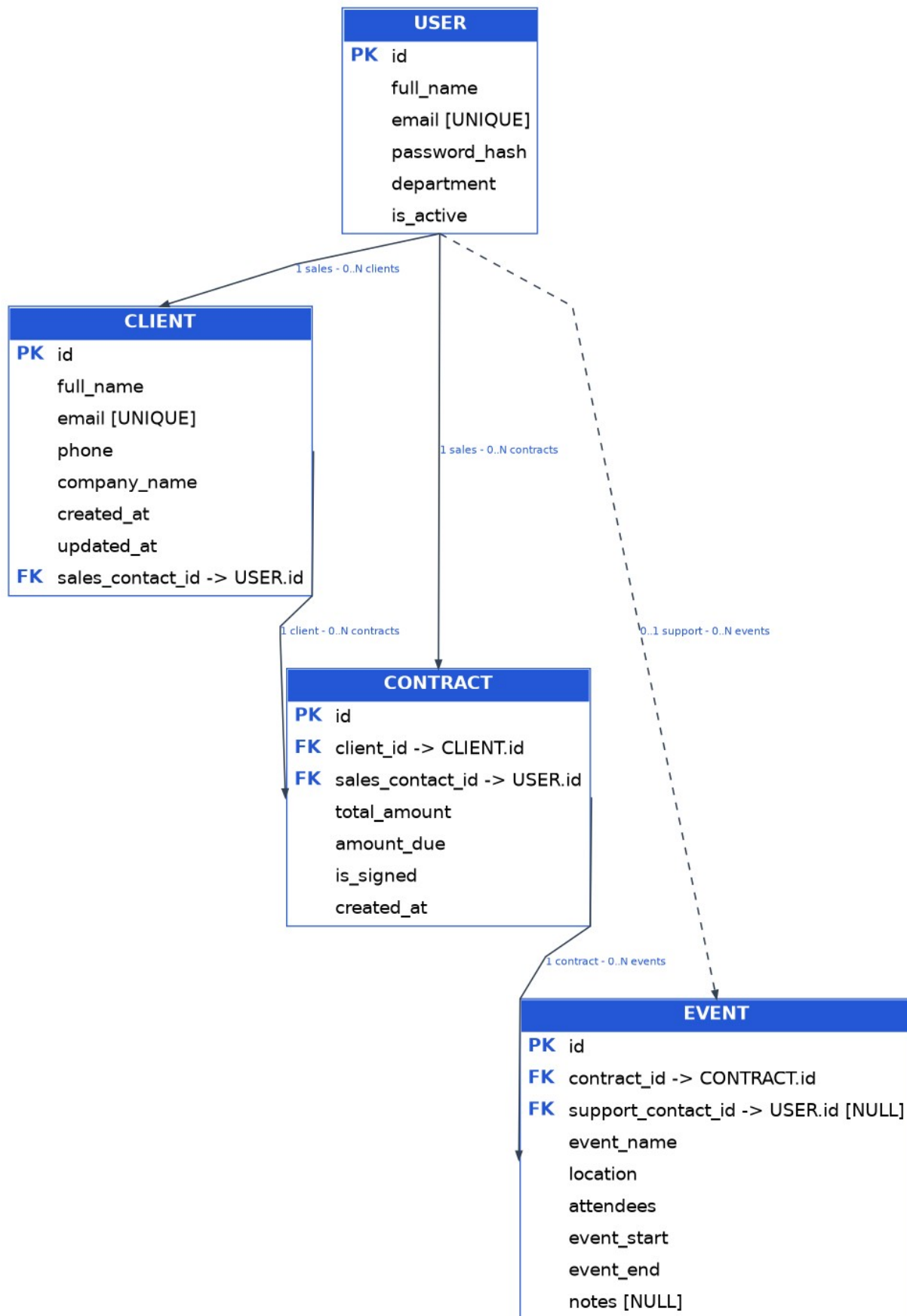


Figure 3 - Final entity relationship diagram

3.2 Tables

users

Field	Type	Constraint	Purpose
id	Integer	Primary key	Collaborator identifier
full_name	Varchar(255)	Required	Display name
email	Varchar(255)	Unique, required	Login identifier
password_hash	Varchar(255)	Required	Bcrypt hash
department	Varchar(50)	Required	MANAGEMENT, COMMERCIAL, SUPPORT
is_active	Boolean	Default true	Logical status

clients

Field	Type	Constraint	Purpose
id	Integer	Primary key	Client identifier
full_name	Varchar(255)	Required	Name
email	Varchar(255)	Unique, required	Email
phone	Varchar(30)	Required	Phone
company_name	Varchar(255)	Required	Company
created_at	Datetime	Automatic	First contact
updated_at	Datetime	Updated	Last contact
sales_contact_id	Integer	FK users, RESTRICT	Responsible sales

contracts

Field	Type	Constraint	Purpose
id	Integer	Primary key	Contract identifier
client_id	Integer	FK clients, RESTRICT	Client
sales_contact_id	Integer	FK users, RESTRICT	Sales contact
total_amount	Decimal(10,2)	Positive	Total
amount_due	Decimal(10,2)	0 to total	Remaining amount
is_signed	Boolean	Default false	Signature
created_at	Datetime	Automatic	Creation

events

Field	Type	Constraint	Purpose
id	Integer	Primary key	Event identifier
contract_id	Integer	FK contracts, RESTRICT	Contract
support_contact_id	Integer	Nullable FK users, SET NULL	Support
event_name	Varchar(255)	Required	Name
location	Varchar(255)	Required	Venue
attendees	Integer	Greater than zero	Attendees
event_start	Datetime	Required	Start
event_end	Datetime	After start	End
notes	Text	Nullable	Notes

3.3 Relationship logic

- One sales collaborator may manage multiple clients and contracts.
- One client may have multiple contracts.
- The contract stores the sales contact associated with the client.
- One contract may cover multiple events.
- An event may be unassigned or have one support collaborator; one support collaborator may manage multiple events.

4. JWT authentication and authorization

4.1 Login and session

The collaborator provides an email to auth login and enters the password through a hidden prompt. After bcrypt verification, the application creates an HS256-signed JWT valid for 60 minutes and stores it in .epic_events_token.json.

```
JWT payload
{
  "sub": "<user_id>",
  "email": "<user_email>",
  "department": "<department>",
  "iat": "<issued_at>",
  "exp": "<expiration>"
}
```

The token is session evidence, not the final permission source. For every protected command, the user is reloaded from SQLite and the account must still exist and be active.

4.2 Authentication versus authorization

Concept	Question	Implementation
Authentication	Who is the collaborator?	bcrypt + JWT + SQLite reload
Authorization	May they perform the action?	require_permission + ownership + assignment in services

4.3 Permission matrix

Operation	Management	Sales	Support
Read clients, contracts, and events	Yes	Yes	Yes
Create / update / deactivate collaborator	Yes	No	No
Create client	No	Yes	No
Update client	No	Own clients	No
Create contract	Yes	No	No
Update contract	All	Own-client contracts	No
Filter unsigned / unpaid contracts	No	Yes	No
Create event	No	Own client + signed contract	No
List events without support	Yes	No	No
Assign support	Yes	No	No
List / update assigned events	No	No	Own assignments
Trigger Sentry test	Yes	No	No

4.4 Deactivation and existing sessions

Functional collaborator deletion is logical deactivation. Related records are preserved. Even when an unexpired token still exists, the SQLite reload immediately rejects a deactivated or deleted account.

5. Security and Sentry monitoring

Risk	Control
Exposed password	Hidden prompt, bcrypt hash, and no password in command history.
Forged or expired token	HS256 signature, exp validation, and invalid-JWT rejection.
Revoked account	User reload and is_active check for every command.
SQL injection	Parameterized Peewee queries; no SQL concatenated with input.
Privilege escalation	Service-layer authorization by role, ownership, and assignment.
Committed secret	Exclusion of .env, JWT_SECRET_KEY, token, local DB, and reports.
Monitoring leak	send_default_pii=False and include_local_variables=False.

5.1 Secret configuration

```
.env
JWT_SECRET_KEY=<long random secret>
SENTRY_DSN=<optional Sentry DSN>
SENTRY_ENVIRONMENT=development
```

The JWT secret and real DSN are never stored in source code. .env and .epic_events_token.json are ignored by Git.

5.2 Sentry architecture

```
src/__main__.py -> initialize_sentry()

Expected business error
-> explicit CLI message

Unexpected technical exception
-> capture_application_exception()
-> Sentry event ID

Controlled demonstration
python -m src monitoring test-sentry
```

The demonstration command is management-only. Without a DSN it stops with an explicit message; with a valid DSN it sends a controlled RuntimeError and returns the Sentry identifier.

5.3 Accepted limits of the local JWT design

- The token is a local bearer token: the operating system must protect the file.
- There is no central revocation list or refresh token.
- These limits fit the scope of a local CLI CRM with short expiration and SQLite revalidation.

6. Testing strategy and coverage

6.1 Organization

```
tests/
├── unit/
│   └── test_monitoring.py
├── integration/
│   ├── test_auth.py
│   ├── test_client_service.py
│   ├── test_contract_service.py
│   ├── test_database.py
│   ├── test_event_service.py
│   ├── test_jwt_auth.py
│   └── test_user_service.py
└── functional/
    └── test_cli.py
```

Category	Scope	Result
Unit	Monitoring isolated with mocks	4
Integration	Auth, JWT, services, models, in-memory SQLite	143
Functional	Public CLI with CliRunner	9
Total	Complete suite	156 / 0 failures

6.2 Validated coverage

Coverage report: 77%
156 tests passed - 0 failures - src/core/jwt_auth.py: 96%

File	statements	missing	coverage
src/__main__.py	27	10	63%
src/cli/auth.py	28	11	61%
src/cli/client.py	35	13	63%
src/cli/common.py	23	6	74%
src/cli/contract.py	56	40	29%
src/cli/event.py	76	46	39%
src/cli/monitoring.py	21	2	90%
src/cli/user.py	32	22	31%
src/core/auth.py	31	1	97%
src/core/config.py	18	1	94%
src/core/database.py	3	0	100%
src/core/jwt_auth.py	52	2	96%
src/core/monitoring.py	20	1	95%
src/models/*	57	0	100%
src/services/client_service.py	50	0	100%
src/services/contract_service.py	71	0	100%
src/services/event_service.py	97	3	97%
src/services/user_service.py	68	0	100%
src/Utils/*	27	27	0%
TOTAL	792	185	77%

coverage.py v7.15.2 - validated on Python 3.12.2 - July 2026

Figure 4 - Updated coverage report

Component	Coverage
Models	100%
Client service	100%
Contract service	100%
User service	100%
Event service	97%
Bcrypt authentication	97%
JWT engine	96%
Monitoring	95%
Total	77%

Initialization utilities remain at 0% because they are executed as installation tools; business services and security modules retain high coverage.

6.3 Validation commands

```
python -m pytest -q
python -m pytest tests/unit -v
python -m pytest tests/integration -v
python -m pytest tests/functional -v
python -m pytest --cov=src --cov-report=term-missing --cov-report=html
```

7. Installation and demonstration

7.1 Clean environment installation

```
git clone https://github.com/AlNocquet/OC-Projet12-EpicEvents.git
cd OC-Projet12-EpicEvents
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python -c "import secrets; print(secrets.token_urlsafe(48))"
# Paste the generated value into JWT_SECRET_KEY
python -m src.utils.create_db
python -m src.utils.create_user "Morgan Manager" manager@epicevents.com
python -m src --help
```

7.2 Authentication workflow

```
python -m src.auth.login manager@epicevents.com
python -m src.auth.check-management
python -m src.client.list
python -m src.auth.logout
```

After login, business commands no longer require the email or password. The token expires after 60 minutes or is removed on logout.

7.3 Suggested ten-minute demonstration

Step	Demonstration	Point explained
1	Architecture and schema	Packages, responsibilities, PK/FK, SQLite, and Peewee.
2	JWT login	Hidden prompt, bcrypt, local token, expiration, and logout.
3	Management	Create collaborator and contract; assign support.
4	Sales	Create client, filters, owned contract, signed event.
5	Support	List and update assigned events.
6	Security	Least privilege, Peewee parameters, secrets, deactivation.
7	Tests	156 tests, 77% total, 96% JWT.
8	Sentry	Controlled exception visible in the dashboard.

8. Final repository verification

```
git status --short
git log --oneline --decorate --graph --all
git ls-files .env .epic_events_token.json epic_events.db
python -m pytest -q
python -m pytest --cov=src --cov-report=term-missing
```

- Clean working tree.
- No secret, JWT token, or local database tracked by Git.
- Reproducible French and English README files.
- Relationship diagram, ADRs, journals, and UML files present.
- Reproducible complete test suite and coverage report.

Conclusion

The current architecture satisfies the CRM functional, technical, and security requirements. It provides a usable JWT session, authorization still grounded in SQLite state, least-privilege permissions, Sentry monitoring, a documented relational schema, and automated validation through 156 tests.